

# CS516: Parallelization of Programs

## Extracting Parallelism from Sequential Programs

**Vishwesh Jatala**

Assistant Professor

Department of CSE

Indian Institute of Technology Bhilai

[vishwesh@iitbhilai.ac.in](mailto:vishwesh@iitbhilai.ac.in)



# Recap

- Introduction to parallelization
- Performance
- Parallel architectures and programming models
- GPU Architectures
- CUDA Programming

**Manual Parallelization!**

---

## Outline for the rest of the course

- Extracting parallelism from sequential programs
  - Dependency analysis
  - Dependency testing
  - Compile-time parallelization techniques
- Paper presentations

# Dependencies

```
S1: A = C + 2;  
S2: B = A * D;  
S3: C = E + D;  
S4: B = 22;
```

- Assume all the variables are initialized to zero
- *Which statements can be executed in parallel?*

# Dependencies

**S1:**  $A = C + 2;$   
**S2:**  $B = A * D;$   
**S3:**  $C = E + D;$   
**S4:**  $B = 22;$

**S1:**  $A = C + 2;$   
**S2:**  $B = A * D;$

**S1:**  $A = C + 2;$   
**S3:**  $C = E + D;$

**S2:**  $B = A * D;$   
**S4:**  $B = 22;$

**S2:**  $B = A * D;$   
**S3:**  $C = E + D;$

**Sequential!**

**Parallel!**

# Bernstein's Conditions

- *When is it safe to run two statements S1 and S2 in parallel?*
  - If none of the following holds:
    - S1 writes into a memory location that S2 reads
    - S2 writes into a memory location that S1 reads
    - Both S1 and S2 write to the same memory location

# Types of Dependencies

- S1 and S2 are two statements
- Read after Write (RAW):
  - *S1 writes first and S2 reads ( $S1 \delta S2$ )*
  - *True or flow dependence*
- Write after Read (WAR):
  - *S1 reads first and S2 writes ( $S1 \delta^{-1} S2$ )*
  - *Anti dependence*
- Write after Write (WAW):
  - *S1 write and S2 writes ( $S1 \delta^0 S2$ )*
  - *Output dependence*

# Types of Dependencies

- S1 and S2 are two statements
- Read after Read (RAR)
  - *S1 reads first and S2 reads*
  - *Input dependence*
  - Not interested, can be executed in parallel!

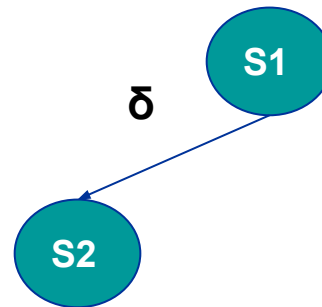


# Types of Dependencies

**S1:**  $A = C + 2;$   
**S2:**  $B = A * D;$   
**S3:**  $C = E + D;$   
**S4:**  $B = 22;$

**S1:**  $A = C + 2;$   
**S2:**  $B = A * D;$

**RAW**  
**(Sequential!)**

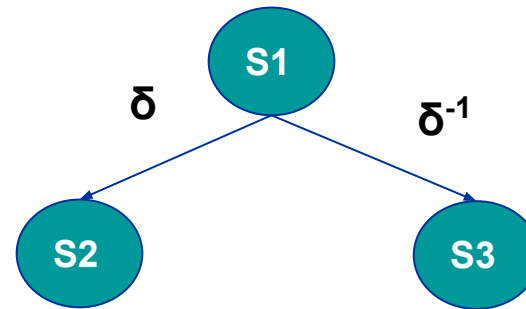


# Types of Dependencies

**S1:**  $A = C + 2;$   
**S2:**  $B = A * D;$   
**S3:**  $C = E + D;$   
**S4:**  $B = 22;$

**S1:**  $A = C + 2;$   
**S3:**  $C = E + D;$

**WAR**  
**(Sequential!)**

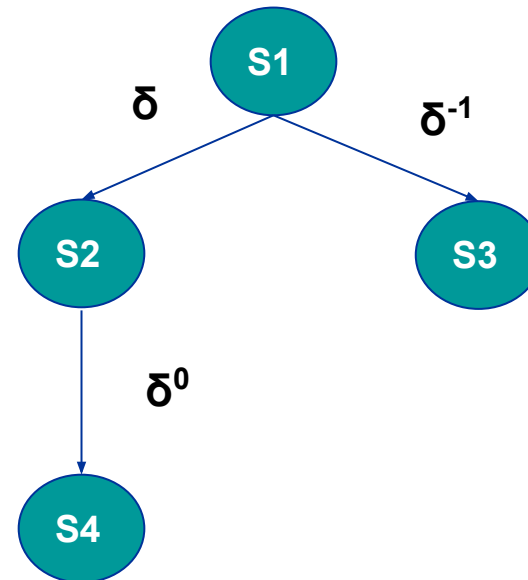


# Types of Dependencies

**S1:**  $A = C + 2;$   
**S2:**  $B = A * D;$   
**S3:**  $C = E + D;$   
**S4:**  $B = 22;$

**S2:**  $B = A * D;$   
**S4:**  $B = 22;$

**WAW**  
**(Sequential!)**



**Data Dependency Graph!**

# Types of Dependencies

**S1:**  $A = C + 2;$

**S2:**  $B = A * D;$

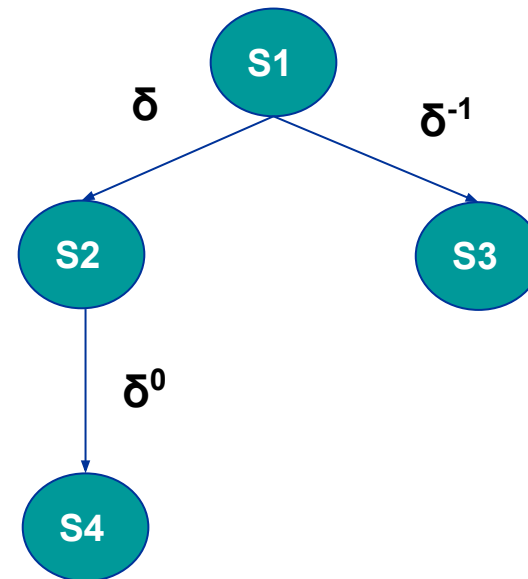
**S3:**  $C = E + D;$

**S4:**  $B = 22;$

**S2:**  $B = A * D;$

**S3:**  $C = E + D;$

**RAR**  
**(Parallel!)**



**S3 and S4?**

**Data Dependency Graph!**

# How to Compute Dependencies?

- *Compute IN and OUT sets for each statement*
  - ▣ *IN set of a statement is the set of variables/memory locations it reads*
  - ▣ *OUT set of a statement is the set of variables/memory locations it writes/modifies*
- *Assuming S2 is reachable from S1*

|                                                     |                     |                        |
|-----------------------------------------------------|---------------------|------------------------|
| $\text{OUT}(S1) \cap \text{IN}(S2) \neq \emptyset$  | $S1 \delta S2$      | Flow (True) dependence |
| $\text{IN}(S1) \cap \text{OUT}(S2) \neq \emptyset$  | $S1 \delta^{-1} S2$ | Anti dependence        |
| $\text{OUT}(S1) \cap \text{OUT}(S2) \neq \emptyset$ | $S1 \delta^0 S2$    | Output dependence      |

# Computing Dependencies

- *Computing dependencies is hard*
  - ▣ *Aliases*
  - ▣ *Irregular accesses*
  - ▣ *Conditional statements*

```
p = a->next
q = b->next
c = *p + b
*q = d + e
```

```
A[b[i]] = a + c
d = A[i] + e
```

```
if (cond)
    b = a + c
a = c + d
```

# Dependency Analysis in Loops

# Loop Dependencies

```
for (i = 1; i <= N; i++)  
S1:  A(i+1) = A(i) + B(i)
```

*Do you see any  
dependency in the loop?*

| i=1                    | i=2                    | i=3                    |
|------------------------|------------------------|------------------------|
| S1: A[2] = A[1] + B[1] | S1: A[3] = A[2] + B[2] | S1: A[4] = A[3] + B[3] |



# Loop Dependencies

```
    for (i = 1; i <= N; i++)  
S1:    A(i+1) = A(i) + B(i)
```

```
    for (i = 1; i <= N; i++)  
S1:    A(i+2) = A(i) + B(i)
```

***What about this loop?***

# Loop Dependencies

```
    for (i = 1; i<=N; i++)  
        for (j = 1; j<=M; j++)  
S1:      A[i+1][j] = A[i][j] + 10
```

*Do you see any dependency in the loop?*

We need a method to formalize the loop dependencies!

# Iterations

```
for (i = 1; i<=6; i++)  
    for (j = 1; j<=5; j++)  
S1:    /*statement*/
```

j



|   |   |   |   |   |   |
|---|---|---|---|---|---|
| * | * | * | * | * | * |
| * | * | * | * | * | * |
| * | * | * | * | * | * |
| * | * | * | * | * | * |
| * | * | * | * | * | * |
| * | * | * | * | * | * |

i

- *The set of all possible iterations*
  - $(1,1) (1,2) \dots (1,5)(2,1)(2,2),\dots(5,5)(6,1)(6,2)\dots(6,5)$

# Iteration Vector

```
for (i = 1; i<=6; i++)  
    for (j = 1; j<=5; j++)  
S1:    /*statement*/
```

j

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| * | * | * | * | * | * |
| * | * | * | * | * | * |
| * | * | * | * | * | * |
| * | * | * | * | * | * |
| * | * | * | * | * | * |

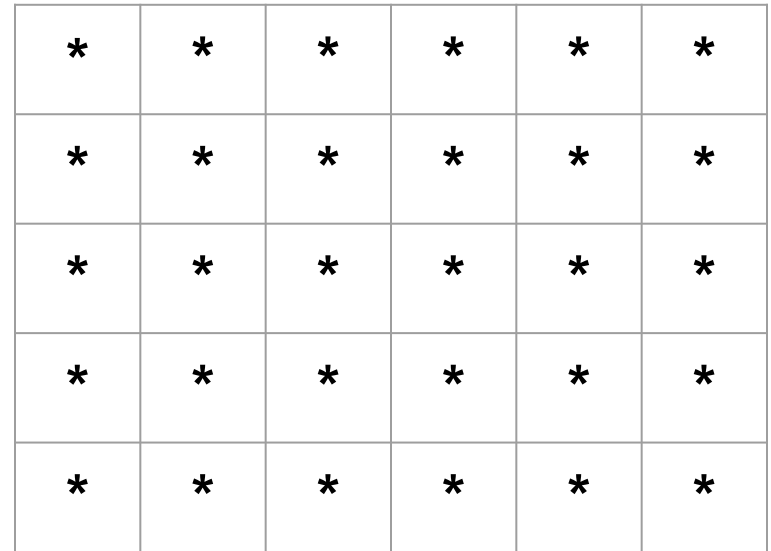
i

- The iteration vector S1[(3, 4)] denotes
  - the instance of S1 executed during the 3rd iteration of the i-th loop and the 4th iteration of the j loop

# Iteration Space

```
for (i = 1; i<=6; i++)  
    for (j = 1; j<=5; j++)  
S1:    /*statement*/
```

j



|   |   |   |   |   |   |
|---|---|---|---|---|---|
| * | * | * | * | * | * |
| * | * | * | * | * | * |
| * | * | * | * | * | * |
| * | * | * | * | * | * |
| * | * | * | * | * | * |
| * | * | * | * | * | * |

i

- Iteration Space: The set of all possible iteration vectors for a statement
- Iteration Space of S1 is  
 $\{(1,1) (1,2) \dots (1,5)(2,1)(2,2),\dots (5,5)(6,1)(6,2)\dots (6,5)\}$

## Ordering of Iteration Vectors

- Useful to define an ordering for iteration vectors
- Define an intuitive, lexicographic order
- Iteration vector  $i$  precedes iteration vector  $j$  ( $i < j$ ), if iteration  $i$  appears before iteration  $j$  in the lexicographic order

Iteration Vector Helps to Formulate Dependencies!

---

## References

- Slides of the book: Randy Allen, Ken Kennedy, Optimizing Compilers for Modern Architectures: A Dependence-based Approach, Morgan Kaufmann, 2001

**Questions?**



**Thank you!**